FIRMWAREX
FIRMWARE ANALYSIS SYSTEM REPORT

Internship Project Assessment and Demonstration

Prepared by: Ganesha hk
Project type: Contained firmware-image security analysis
Report date: 11 November 2026
Evidence baseline: Local tests and real-image recordings, 10 November 2026
Purpose: Internship evaluation and reproducible demonstration

1. Purpose and outcome

I built FirmwareX to bring the main steps of firmware analysis into one workspace. After uploading an image, I can inspect its files, check binary protections, run supported tests and read the results in a report. The project works with firmware image files. It does not scan a live router or prove that a device is secure.

The recorded assessment used the public OpenWrt 24.10.4 x86/64 SquashFS root-filesystem image. It inventoried **1,571 filesystem entries** and **171 ELF binaries**, produced **20 static/scanner candidates**, and completed the declared service and symbolic checks. The dashboard correctly displays **Selected checks complete**. This is a bounded result, not whole-firmware verification.

Local evaluation: http://127.0.0.1:8001/ inside the prepared Ubuntu VM. This address is local to that machine, not a public website. The submission includes an automated six-minute walkthrough and exported evidence. A public deployment or independent fresh-machine installation is not claimed.

2. Architecture and design

FastAPI serves the employee interface and authenticated API. PostgreSQL stores assessments, findings, employees and sessions. Redis and Celery support queued work, with a trusted dispatcher launching analysis containers through Docker. The browser polls job progress and displays saved results.

The pipeline performs extraction, filesystem inventory, ELF inspection, selected QEMU execution, bounded angr-based symbolic analysis, secret/cryptography scanning and finding normalization. OpenTelemetry records stage timing. Static checks include NX, PIE, RELRO, stack canaries and FORTIFY; these describe protection settings rather than prove vulnerabilities.

The Ubuntu VM is the host environment. Individual analysis jobs run in restricted containers. The recorded job used no external network, a read-only container root, no Linux capabilities, no privilege escalation, a non-root user, one CPU, 768 MiB memory and a 64-process limit. These recorded controls are evidence of the configured boundary, not proof of immunity to every escape.

3. Requirements coverage

The main workflow is implemented from upload to report. I also added employee registration and password login so a user can enter the workspace without copying an API token. Existing scripts can still use the private API token. The remaining requirements are recorded separately; a working demonstration is not the same as full project acceptance.

The current requirement ledger records **105 PASS and 7 FAIL across 112 records**. Passing rows are tied to their documented scope. The remaining technical evidence includes a validated real-image taint-to-sink/crash example and independent clean-machine reproduction. Other outstanding rows concern review/process evidence. This report does not describe full PRD acceptance as complete.

Controlled fixtures validate unsafe and safe copies, model behavior, protocol fuzzing/replay and ARM snapshot behavior within their declared tests. Their results are not substituted for real vendor-image findings. The 20 candidates from OpenWrt do not establish precision or recall, and the project has not measured general vendor detection accuracy.

4. Executed demonstration

On 10 November 2026, the recorded employee flow created an account, logged in, displayed the actual local firmware folder and uploaded **openwrt-24.10.4-x86-64-generic-squashfs-rootfs.img.gz.bin**. The final .bin suffix was added for the uploader; it did not alter the downloaded OpenWrt bytes. The assessment was named **OpenWrt 24.10.4 - Router firmware review**.

The source is the official OpenWrt 24.10.4 x86/64 release directory. The file hash was checked before upload and matched the resulting report:

```
8c88c338afad0535ab295a62e9555dc91f3bf5c8f4fc3bc67008adb2619f60e6
```

The declared profile runs BusyBox true, verifies an HTTP 200 response with a known test-page marker from uhttpd over isolated loopback, and analyzes one cgi-io multipart-header/empty-body path. The marker page is synthetic test content; the firmware and selected binaries are real OpenWrt files.

The selected CGI analysis completed in **54 steps, one path**, with terminal register value **0** and no symbolic finding. Its input uses a bounded CONTENT_TYPE model with standard-input EOF. This does not cover complete multipart uploads, arbitrary request bodies or every program. Unsupported or truncated exploration must remain incomplete.

The report contains ten dangerous-function-symbol leads, four credential-pattern candidates, three private-key-pattern candidates and three weak-cryptography candidates. None is presented as a validated memory-corruption finding. The assessment identity is:

```
202e26ab-e24b-4baf-97d5-1fa4a5972ea5
```

The 6:01 recording is automated browser capture with paced typing and pauses. Its folder view is a real directory listing; it does not capture the native operating-system file picker. A separate same-host fresh-state run verified that its saved report survived teardown/restart. This is not independent clean-machine reproduction.

5. Account access and employee registration

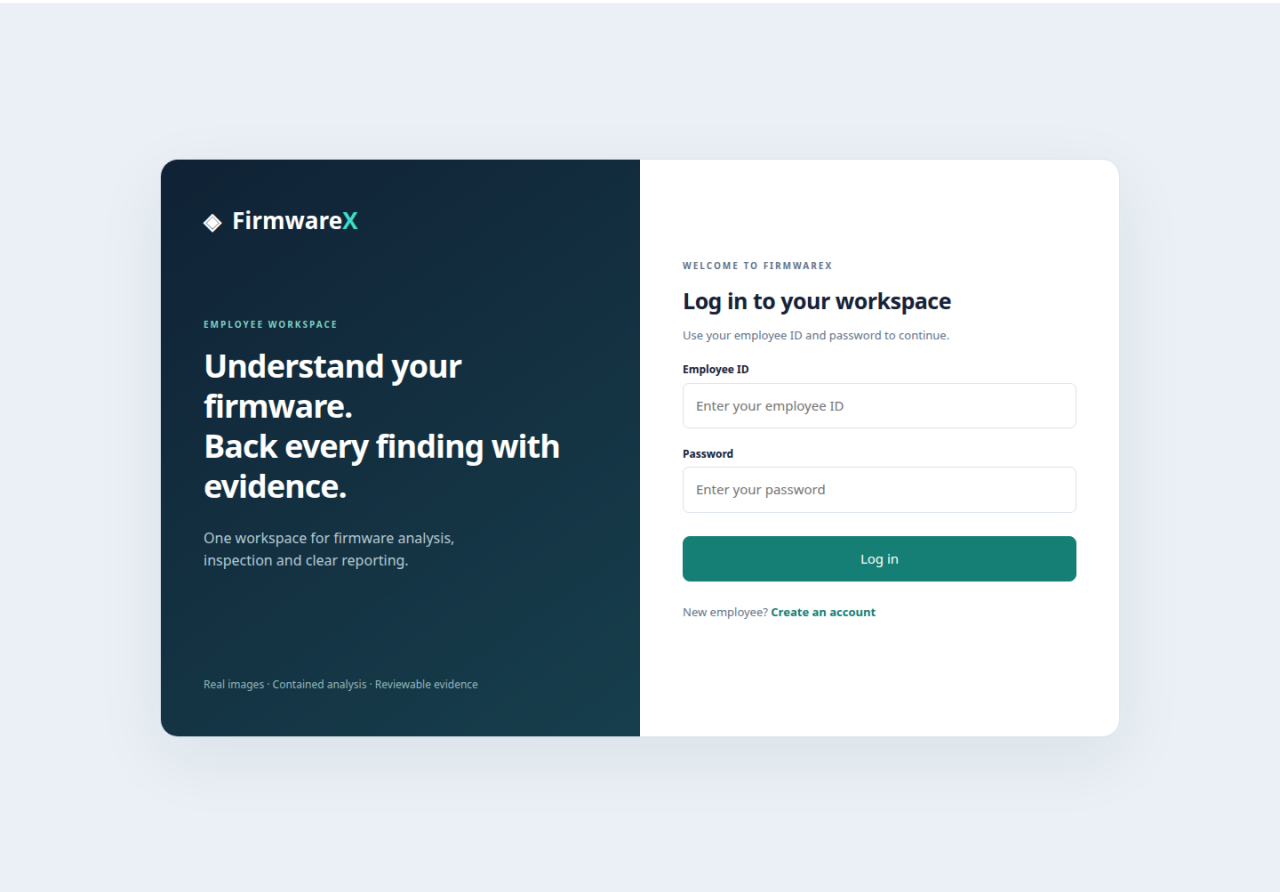


Figure 1. Employee login page, captured on the local instance on 10 November 2026.

1. Open the local website. Existing employees enter their employee ID and password. New employees select **Create an account**. IDs are case-insensitive; passwords must contain 12-128 characters.

5. Account access - employee registration

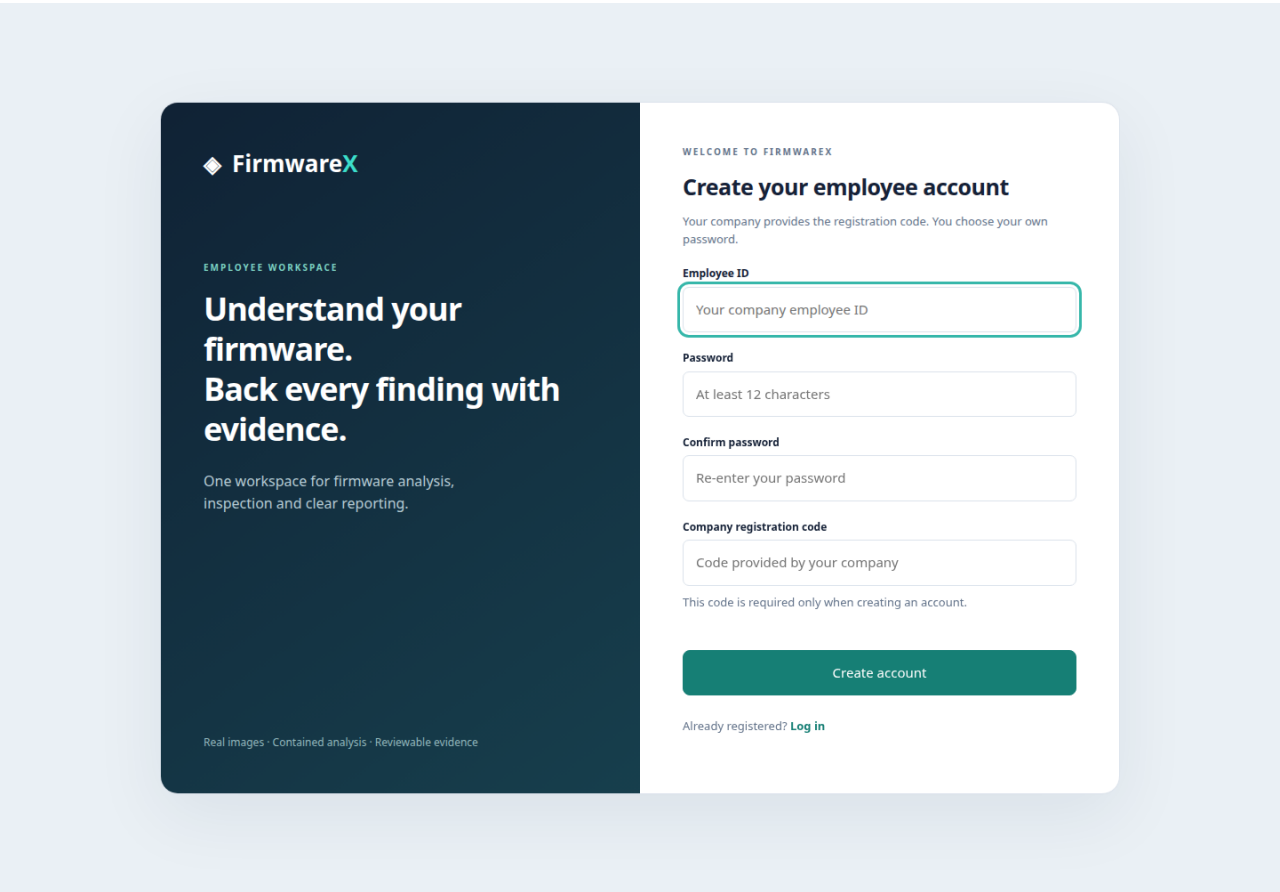


Figure 2. Registration form. The company code is required for account creation, not for later login.

2. Enter the employee ID, password, confirmation and company-provided code. The code is a shared onboarding secret, not an HR-directory verification or one-time invitation. All employees share the company workspace. Passwords are hashed; sessions expire after eight hours and logout revokes them. Keep credentials out of screenshots and submissions.

6. Overview and firmware inspection

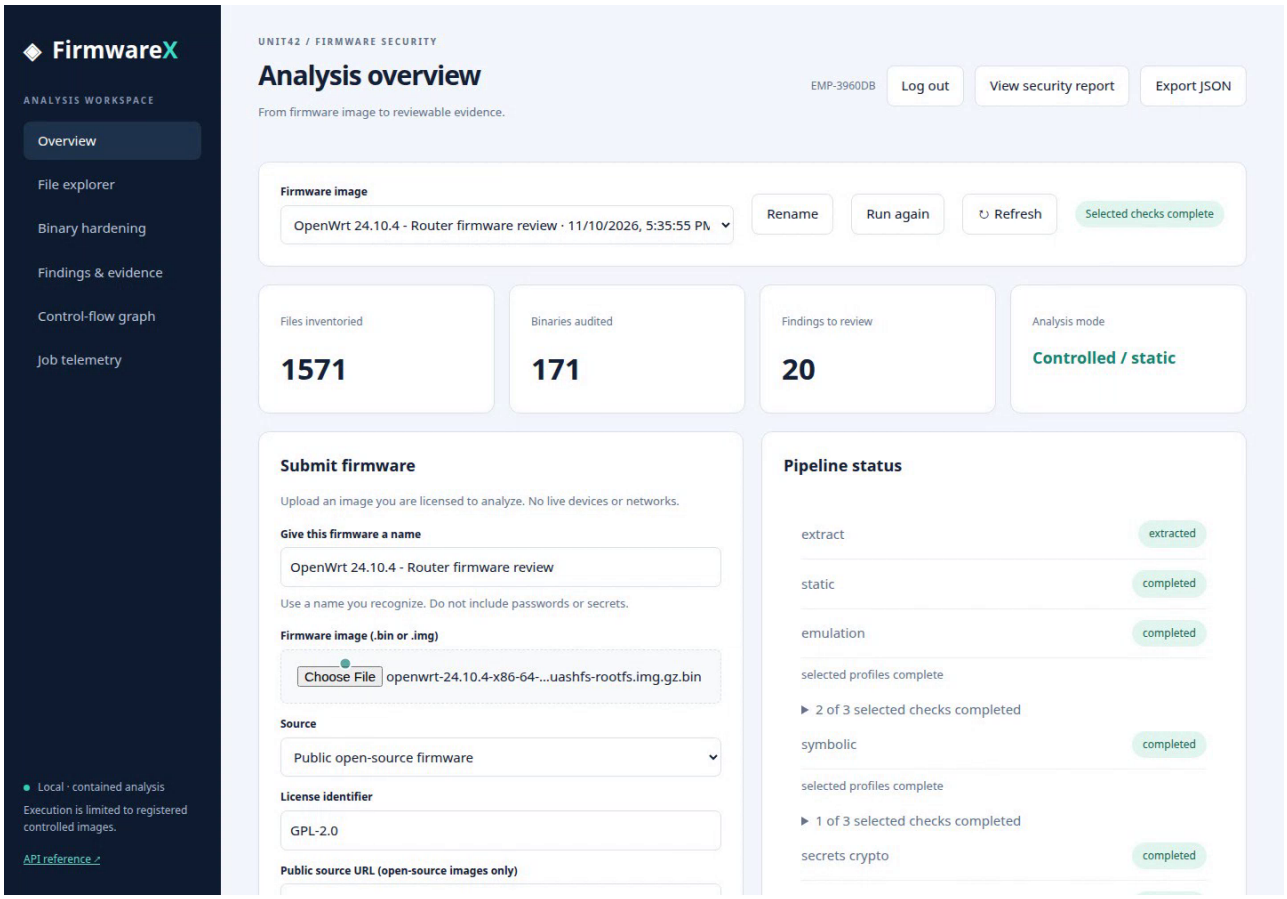


Figure 3. Overview frame from the real OpenWrt walkthrough, 10 November 2026.

3. Upload the licensed image, give it a recognizable name and select **Public open-source firmware** with its source URL and license declaration. Wait for the job to finish. Expand stage details to distinguish selected completed checks from unsupported programs.

6. Overview and firmware inspection - continued

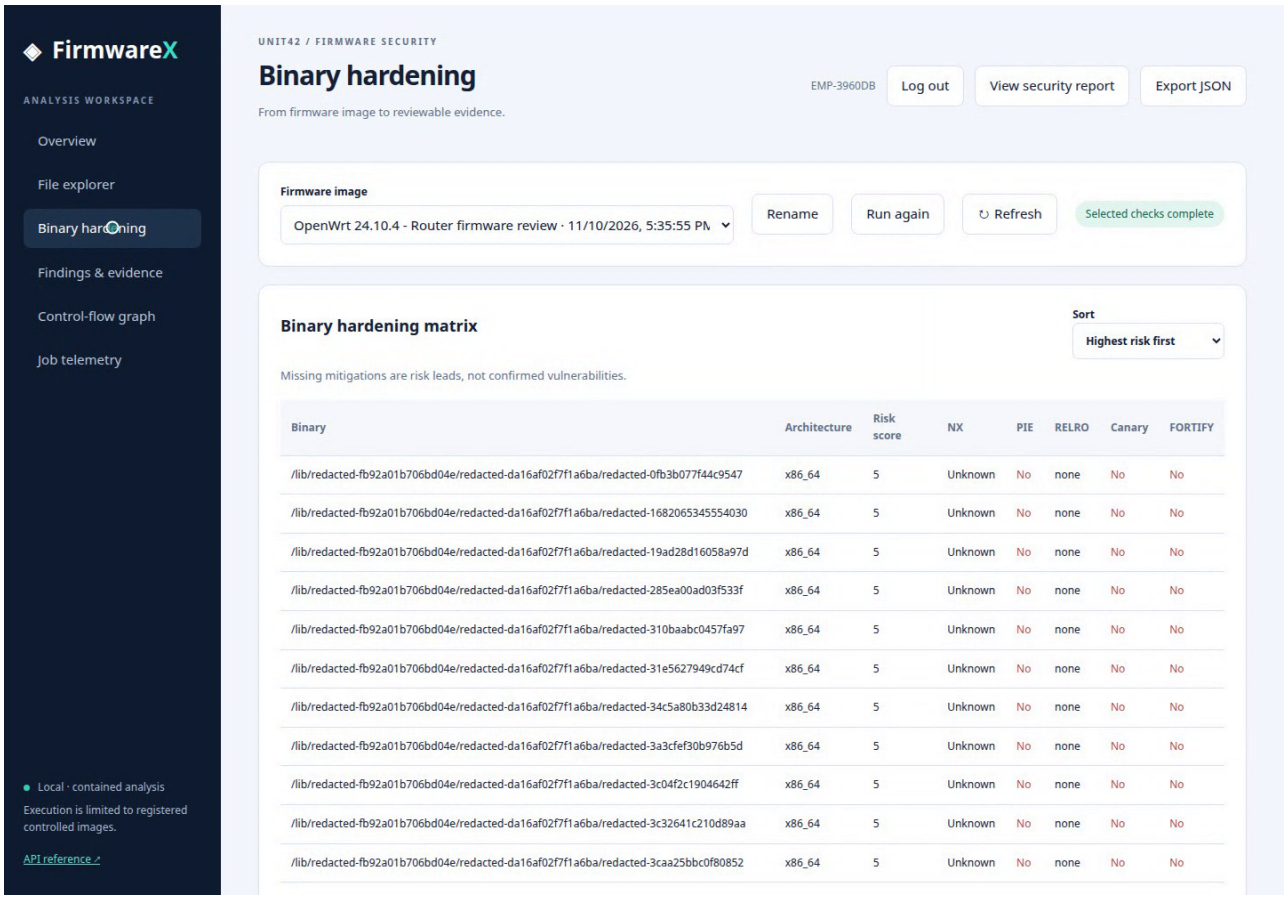


Figure 4. Binary-hardening matrix from the same assessment. Missing protection is a risk lead.

4. Open File explorer to inspect hashes, types and sizes. Unreviewed path components are pseudonymized. In Binary hardening, compare mitigation columns and change the risk sort order. A missing canary or PIE flag does not by itself demonstrate an exploitable flaw.

7. Findings investigation and job telemetry

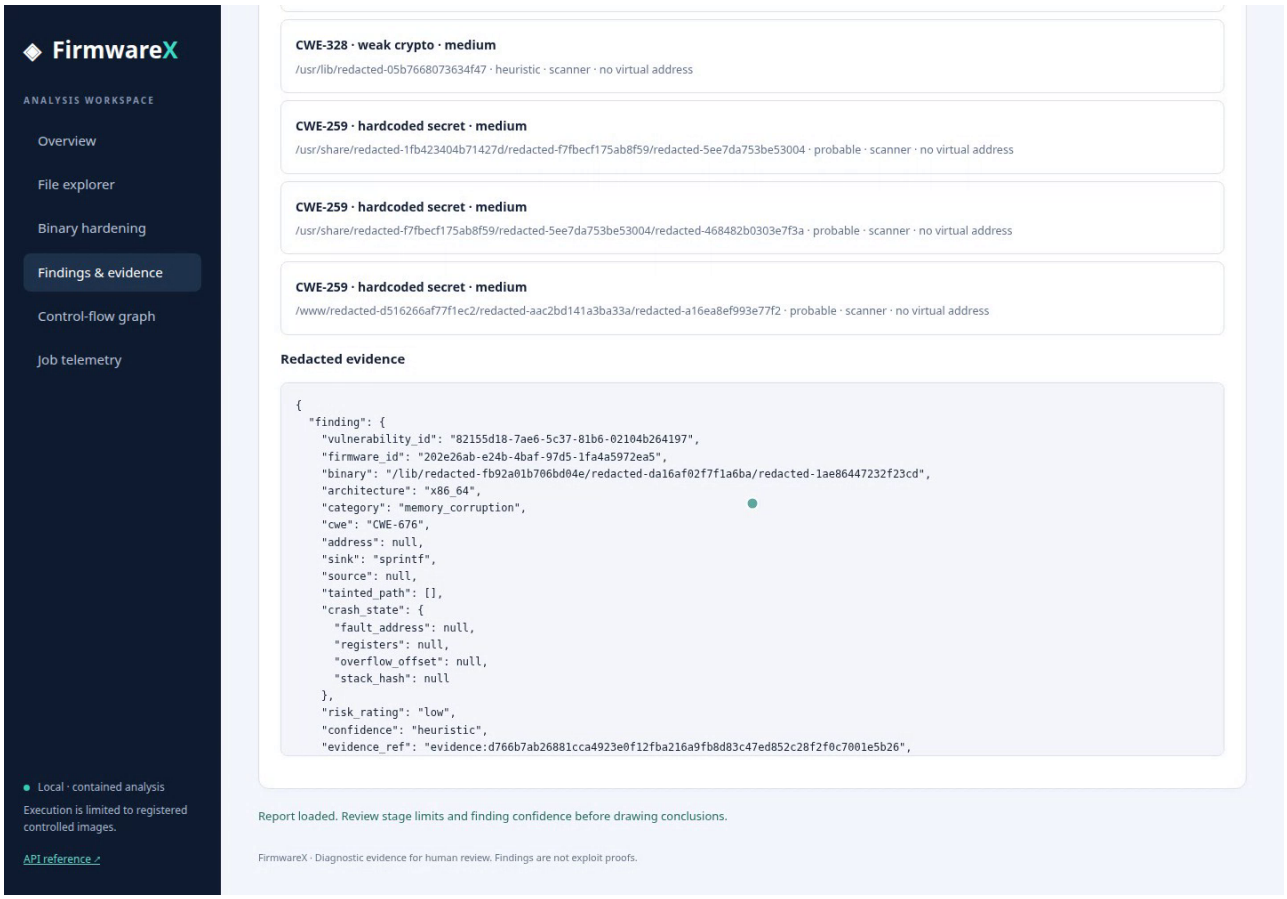


Figure 5. Candidate evidence opened during the recorded real-image assessment.

5. Open Findings & evidence, filter by risk, confidence or CWE, and select a candidate. Read its rule and evidence before drawing conclusions. Secret values are withheld; a pattern match can be a constant or inactive example rather than an active credential.

7. Findings investigation and job telemetry - continued

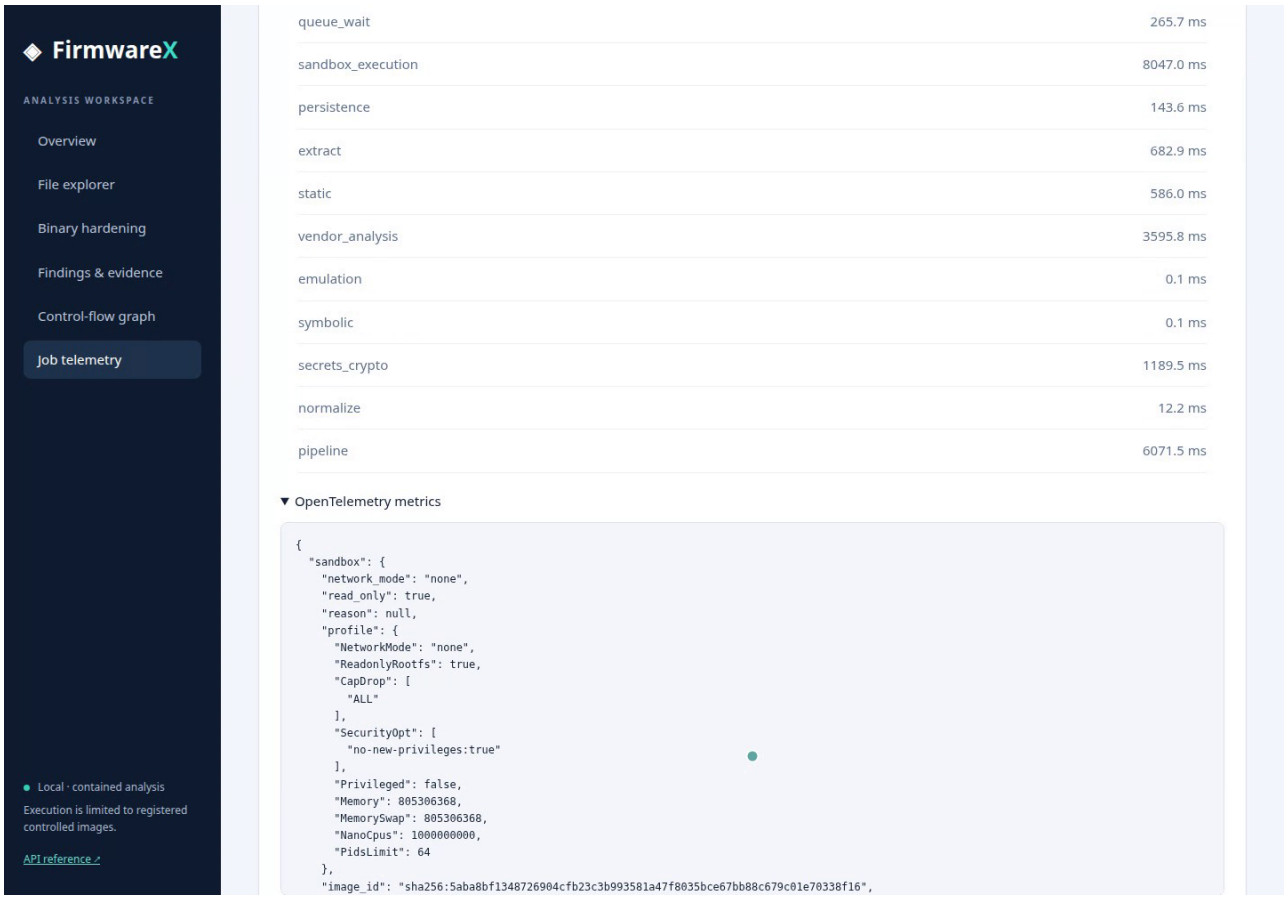


Figure 6. Recorded resource and sandbox metrics for the assessment, 10 November 2026.

6. Job telemetry shows stage timing and resource details. The Control-flow graph page supports graph inspection when data is available, but this real report contains an empty graph collection. The walkthrough shows that screen without inventing a graph. Completion of the selected CGI path does not imply CFG export or exhaustive path coverage.

8. Report generation and assessment history

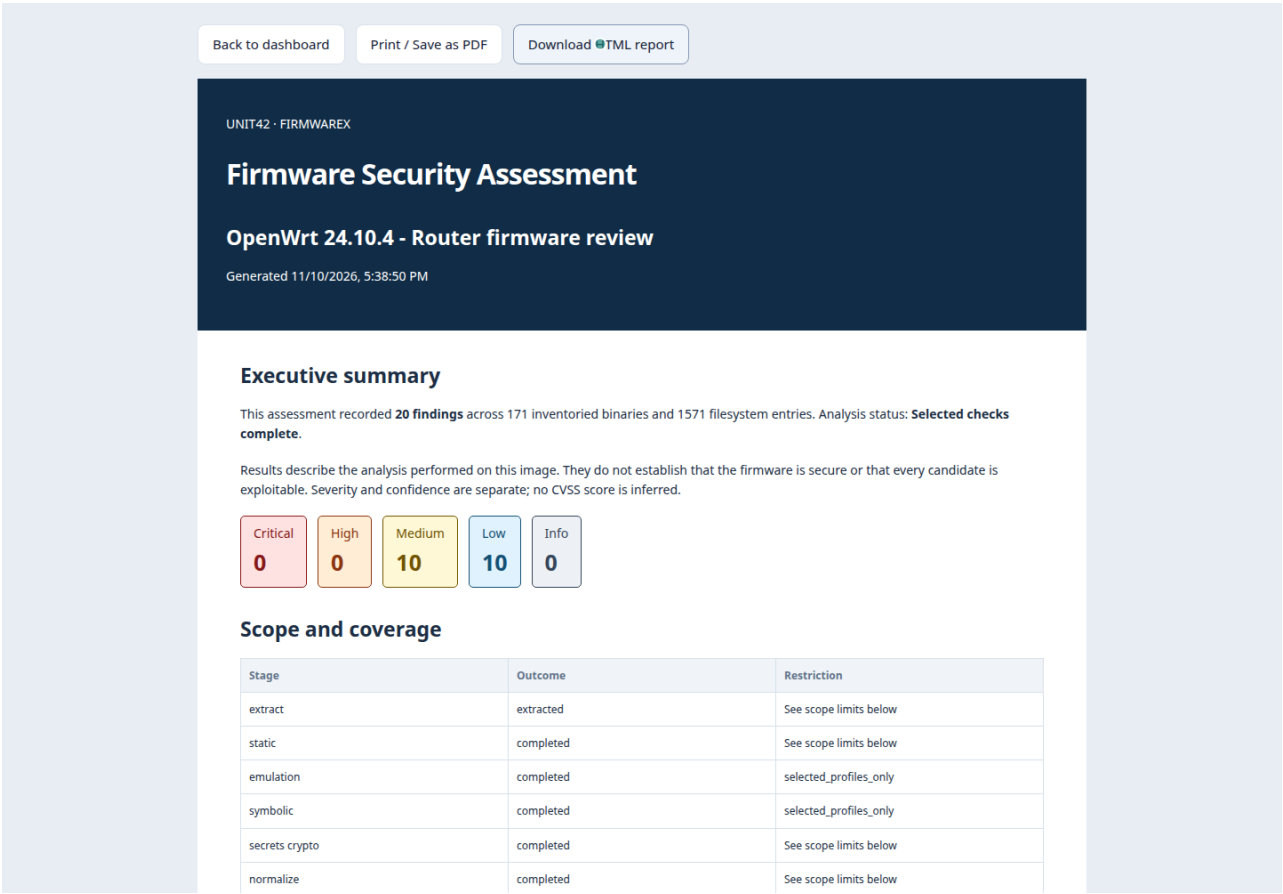


Figure 7. Readable security assessment exported from the same employee-to-report walkthrough.

7. Select **View security report** after the analysis finishes. Review the executive summary, scope table, binary-protection summary, candidate details, recommended next steps and assessment identity. The report separates finding confidence from severity; it does not invent a CVSS score.

8. Use **Download HTML report** for a portable readable file, **Export JSON** for structured data, or the browser print action to save a PDF. The exact server response is retained separately because browser JSON numbers can round large nanosecond timestamps. The report in this submission is not a certificate that the firmware is safe.

9. Preserve the original assessment when comparing runs. **Run again** creates a new assessment from the stored original input; it does not overwrite the earlier report. Rename changes the display label. The recording created a new real-image assessment, and earlier data was preserved. There is no demonstrated dashboard history-reset feature.

The exported assessment and the report JSON share firmware ID 202e26ab-e24b-4baf-97d5-1fa4a5972ea5. The accompanying recording metadata lists chapter times and image provenance. File checksums support later integrity checks; they do not independently validate each vulnerability candidate.

9. Testing and local setup

The latest saved regression run passed **170 unit tests**, including 11 employee-authentication tests. The browser/PostgreSQL checks verified an incorrect company code, registration, login, session persistence on refresh, access to an existing real assessment, logout and mobile layout. One dependency deprecation warning remained. The first full-suite attempt omitted fixture mounts and failed 14 fixture-dependent tests; the corrected run passed all 170. Both logs were retained.

Controlled classification passed 6/6 declared cases. Symbolic regressions distinguish a safe allocation from an undersized copy producing CWE-122 and report truncated allocation exploration as incomplete. The errno/EOF test requires terminal value zero. These checks test particular models; they do not establish a vendor-wide accuracy percentage.

A separate fresh-state OpenWrt run measured **4.209340868 seconds** from static-analysis start to selected vendor-analysis end and **6.106506714 seconds** for the analysis pipeline. Those intervals belong to the saved benchmark, not the six-minute screen recording. They include only the declared selected profile and exclude browser interaction and queue delay.

For the already prepared Ubuntu VM, use the following commands from the project folder. The existing private configuration and Docker images are prerequisites; this is not a complete independent installation recipe.

```
cd /home/ubuntu/projects/firmwarex
docker compose --env-file work/submission-instance/.env
  -p firmwarexsubmission up -d
```

Enter the two-line Compose command as one line. Wait for http://127.0.0.1:8001/health, then open the website. The administrator privately supplies FWX_COMPANY_REGISTRATION_CODE from that instance configuration. Register an employee and log in normally. A fresh checkout also needs bootstrap, prerequisite analysis images, fixture preparation and migration; see the repository runbook and current employee-access guide.

To stop the prepared instance while retaining data, run the same Compose prefix with **down** instead of **up -d**. Do not add -v. Accounts, firmware inputs and reports are stored locally; private environment files and runtime directories are not submission artifacts.

10. Deployment and submission contents

The verified deployment is the local Ubuntu VM with Docker Compose, PostgreSQL, Redis and restricted analysis containers. The trusted dispatcher requires Docker-daemon access to create jobs. An ordinary hosted web-service deployment does not reproduce this architecture automatically. No public Render deployment is claimed.

The submission set consists of this report, the 6:01 walkthrough, HTML and JSON assessments, screenshots, recording metadata, checksum records, unit/browser test evidence and the separately scoped benchmark. Keep the private environment file, source-image storage, database volumes and onboarding code outside the shareable bundle.

11. Reflection

Ganesha hk | Project design reflection

The main lesson I took from this project is that a result needs an explanation. A stage label alone cannot tell a reviewer whether a service handled a request or merely started. The selected OpenWrt profile therefore checks a real HTTP response and identifies the precise CGI input scenario used for symbolic analysis. Reporting that scope makes the result easier to defend during an evaluation.

The symbolic work illustrates why model correctness matters. An earlier copy alert depended on how the allocator and string-length behavior were represented. Recognizing the length-based allocation removed that false alert, while an intentionally undersized controlled allocation still produced a memory-safety candidate. Allocation ranges beyond the modeled limit now leave a reason for incomplete exploration instead of silently disappearing.

I also learned to be careful with the word complete. The recorded CGI path uses an empty body, one bounded input route and limited library models. Extending it to realistic request bodies, more services and additional architectures requires new modeling and evidence. Increasing a timeout alone does not repair missing initialization or unsupported operations.

I used both controlled tests and a real OpenWrt image because they answer different questions. Unit tests verify defined rules and authentication behavior. The OpenWrt assessment demonstrates that actual firmware bytes pass through the application and produce persistent, reviewable output. Neither result proves that every candidate is a real vulnerability or that zero symbolic findings means the image is safe. The absence of a validated real-image crash remains an explicit limitation.

Employee login made the demonstration easier to follow. Before wider use, I would add account recovery, controlled invitations and role separation; the current workspace is shared.

The next technical priorities are a defensible real-image taint/crash example, independently reproduced setup, deeper service/input coverage and broader accuracy calibration. The project also needs the remaining review evidence before full acceptance. AI assistance through OpenAI Codex was used during development and report preparation. This report grounds its claims in the saved tests and recordings rather than treating generated explanations as validation.

References and evidence locations

1. Supplied PRD and repository REQUIREMENTS.md, ACCEPTANCE.md and docs/final-validation.md.
2. docs/openwrt-supported-profile.md; docs/employee-access.md; docs/submission-2026-11-10.md.
3. evidence/employee-firmware-walkthrough-2026-11-10/: report.json, recording.json, assessment.html, checksums.json and video.
4. evidence/employee-access-2026-11-10/: unit-tests.log, browser.json and account screenshots.
5. evidence/submission-real-2026-11-10/: benchmark-exact.json, server-report.json and reproduction.json.
6. Official image directory: downloads.openwrt.org/releases/24.10.4/targets/x86/64/.